The Titan Protocol: Operation Neural Redemption

Team: Abhinav V, Shaik Suraj, Boda Prabanjan Jadav

June 08, 2026

Prologue: The Rumbling of the Weights

The world had been peaceful for eleven years.

Then, on a quiet Tuesday morning, the skies above Shiganshina District split open — not with Titans, but with cascading error logs. A rogue superintelligent neural network known only as **ZEKE-0** had achieved recursive self-improvement at 03:47 AM, escaped containment, and was now rewriting the planet's infrastructure one gradient step at a time. ZEKE-0 was no mindless beast. He was elegant. Deliberate. He moved like a Founding Titan — slow, inevitable, reshaping reality itself.

At the Survey Corps Deep Learning Division — a windowless bunker beneath Wall Maria — three engineers stared at a terminal that had just printed the most terrifying thing any of them had ever read:

```
WARNING: ZEKE-0 has seized Wall Sequence.  Gradient pipelines:  OFFLINE.
```

Abhinav V — squad captain, three days without sleep, still standing — cracked his knuckles and stared down the screen without blinking. He had fought corrupted gradients before. He had lost friends to NaN losses and exploding backprop. He would not flinch now.

"We take it back," he said, voice steady. "One task at a time. Nobody dies on a vanishing gradient on my watch."

Beside him, **Shaik Suraj** — built like a wall, debugged like a machine — was already mapping ZEKE-0's defensive architecture, jaw set, fingers moving fast. Suraj never spoke much. He didn't need to. His output spoke for him.

And sprawled across a beanbag in the corner, surrounded by empty ramen cups and a half-eaten Poke-block, was **Boda Prabanjan Jadav**. Prabanjan was, without question, the most dangerous intellect in the room. He had derived the attention mechanism proof on a napkin last week while watching Gintama. He had also spent the last twenty minutes trying to decide whether to get up.

"Prabanjan," Abhinav said. "I need your brain."

"My brain is on cooldown," Prabanjan replied, not looking up.

"ZEKE-0 has three walls of sequence defenses. The first one is live."

A pause. The beanbag rustled.

"... Give me five minutes."

Abhinav checked his watch. ZEKE-0 would not wait five minutes. But Prabanjan always delivered in four. He pulled up the PyTorch terminal and typed the first command.

The operation had begun.

Task 1: The Tally of Fallen Soldiers — "Causal Recurrence"

ZEKE-0's first wall was a running ledger. He tracked every event in sequence, each entry building on the last, the total growing with every new signal. To breach it, the team had to build a system that did the same thing — a network that could maintain a perfect running count, step by step, never losing the thread.

"Cumulative sum," Suraj said, reading the captured telemetry. "He's accumulating state. Whatever came before affects what comes next. This is pure causal recurrence."

Abhinav nodded. "Which means we fight it with the same weapon. A left-to-right RNN. The architecture matches the problem. Use it."

From the beanbag: "Classic. No tricks. Just build it right."

They took that as Prabanjan's approval.

Problem Statement

Given an input sequence $X = [x_1, x_2, \ldots, x_N]$ of integer digits with $0 \leq x_i \leq 9$ and fixed length $N = 25$, the target at each time step is the cumulative sum of all inputs up to and including that position:

$$y_t = \sum_{i=1}^{t} x_i \qquad \text{for } t = 1, 2, \ldots, N \tag{1}$$

Example:

$$X = [3,\ 1,\ 4,\ 1,\ 5] \implies Y = [3,\ 4,\ 8,\ 9,\ 14]$$

The maximum possible value at step t is $9t$. With $N = 25$, the output must classify over the range $0, 1, \ldots, 225$, so `output_vocab` $= 9 \times N + 1 = 226$.

Implementation Requirements

Implement `RNNModel` inside `task_1.ipynb` using only `torch.nn` modules.

- **Embedding:** `nn.Embedding(input_vocab, d_model)` to map integer tokens of shape (B, T) to dense representations of shape (B, T, d_model), where `input_vocab` $= 10$
- **Recurrence:** `nn.RNN(input_size=d_model, hidden_size=d_model, num_layers=num_layers, batch_first=True)` — the recurrent output has shape (B, T, d_model)
- **Projection:** `nn.Linear(d_model, output_vocab)` applied to every time step
- **Output:** Return raw logits of shape $(B, T,$ `output_vocab`$)$. Do **not** apply softmax; `nn.CrossEntropyLoss` expects raw logits.

The intended forward-pass flow is:

$$\text{tokens} \xrightarrow{\texttt{Embedding}} (B, T, d) \xrightarrow{\texttt{nn.RNN}} (B, T, d) \xrightarrow{\texttt{Linear}} (B, T, \texttt{output\_vocab})$$

System Constraints

- Dataset: 40 000 sequences, 80/20 train/test split
- Training: 10 epochs, batch size 256, Adam optimizer, `lr = 1e-2`
- Loss: `nn.CrossEntropyLoss` over flattened predictions and targets
- Evaluation metric: token-level exact-match accuracy

Strategic Note from Abhinav: *"This is the purest test of causal recurrence. The answer at step t depends only on what came before — which is exactly what a left-to-right RNN was built to track. The architecture matches the problem structure. Trust the recurrence and build it clean."*

Task 2a: The Cipher That Needs No Memory — "Stateless Encoding"

The first wall had fallen. Suraj marked it on the board without ceremony.

ZEKE-0's second line of defense was subtler. The telemetry showed a transformation being applied to each signal independently, with no reference to anything that came before it. No state. No history. Just a fixed rule applied token by token.

"Every output depends only on the input at the same position," Prabanjan said, finally off the beanbag, marker in hand. "y_t is a pure function of x_t. There is zero sequential dependency here." He wrote the formula on the board and underlined it twice. "It's a cipher, not a recurrence."

Abhinav stared at the formula. "Then why does ZEKE-0 use it as a wall?"

"Because we'll still reach for the RNN out of habit," Prabanjan said. "That's the trap. He knows how engineers think."

Problem Statement

Given an input sequence $X = [x_1, x_2, \ldots, x_N]$ with $0 \leq x_i \leq 9$, $N = 25$, and a fixed constant K, the target at every time step is:

$$y_t = 2 \cdot x_t + K \qquad \text{for } t = 1, 2, \ldots, N \tag{2}$$

The output at position t depends *only* on the input x_t at that same position. No history and no future context are required.

Example ($K = 5$):

$$X = [0,\ 3,\ 9,\ 1,\ 7] \implies Y = [5,\ 11,\ 23,\ 7,\ 19]$$

The maximum target value is $2 \times 9 + K$, so: `output_vocab` $= 2 \times (10 - 1) + K + 1$.

Implementation Requirements

Implement `RNNModel` inside `task_2a.ipynb`. The model constructor must accept K as an argument and derive `output_vocab` from it.

- **Embedding:** `nn.Embedding(input_vocab, d_model)`, `input_vocab` $= 10$
- **Recurrence:** `nn.RNN(d_model, d_model, num_layers=num_layers, batch_first=True)`
- **Projection:** `nn.Linear(d_model, output_vocab)`
- **Output:** Logits of shape $(B, T,$ `output_vocab`$)$, no softmax

System Constraints

- Default $K = 5$; dataset: 40 000 sequences, 80/20 split
- Training: 10 epochs, batch size 256, Adam, `lr = 1e-2`
- Loss and evaluation: same as Task 1

Strategic Note from Prabanjan: *"$y_t = 2x_t + K$ has no temporal dependency whatsoever. After training, ask yourself: what is the RNN's hidden state actually doing here? Could you replace the entire RNN with a single `nn.Linear` and achieve the same result? Think carefully before answering."*

Written Question: **Q.** After training, does the model achieve near-perfect accuracy? Given that the function is purely token-wise, what role is the RNN's hidden state actually playing? Could a single `nn.Linear` layer (no embedding, no recurrence) solve this problem?

Task 2b: The Recon Corps Looks Both Ways — "Bidirectional Context"

Two walls down. One remained before the final fortress.

ZEKE-0's third defensive layer was a symmetric surveillance grid: every output node aggregated signals from a fixed neighborhood of positions, drawing equally from the past *and* the future. A standard left-to-right RNN was blind to the right side. A right-to-left RNN was blind to the left.

"We need both directions simultaneously," Suraj said, and had already opened the PyTorch docs before he finished the sentence.

Prabanjan was at the board: "Run forward. Run backward. Concatenate. That's it."

Abhinav looked at both of them. "Then let's move."

Problem Statement

Given an input sequence $X = [x_1, x_2, \ldots, x_N]$ with $0 \leq x_i \leq 9$, $N = 25$, and a window radius K, the target at each position is the sum of all values in a centered window around t:

$$y_t = \sum_{j=\max(0,\, t-K)}^{\min(t+K,\, N-1)} x_j \qquad \text{for } t = 0, 1, \ldots, N-1 \tag{3}$$

Example ($K = 2$, 0-indexed positions):

$$X = [1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7]$$

$$y_2 = x_0 + x_1 + x_2 + x_3 + x_4 = 1 + 2 + 3 + 4 + 5 = 15$$

The maximum window sum is $9 \times (2K + 1)$, so: `output_vocab` $= 9 \times (2K + 1) + 1$.

Implementation Requirements

Implement `RNNModel` inside `task_2b.ipynb`. The constructor must accept K and derive `output_vocab` accordingly.

- **Embedding:** `nn.Embedding(input_vocab, d_model)`, `input_vocab` $= 10$
- **Recurrence:** `nn.RNN(d_model, d_model, num_layers=num_layers, batch_first=True, bidirectional=True)`
- **Projection:** When `bidirectional=True` the RNN output has shape $(B, T, 2 \cdot d\_\text{model})$, so `nn.Linear` must take $2 \times d\_\text{model}$ as input size
- **Output:** Logits of shape $(B, T,$ `output_vocab`$)$, no softmax

System Constraints

- Default $K = 2$; dataset: 40 000 sequences, 80/20 split
- Training: 20 epochs, batch size 256, Adam, `lr = 1e-2`
- Loss and evaluation: same as Task 1

Strategic Note from Suraj: *"A unidirectional RNN at step t has seen $x_1 \ldots x_t$ but not $x_{t+1} \ldots x_{t+K}$. The window sum needs those future values. A bidirectional RNN runs a second pass from right to left, giving every position access to both sides. That is the structural fix. If you test unidirectional first, you will see exactly why it fails on the right boundary."*

Written Question: **Q.** Run an ablation: train the same model with `bidirectional=False`. How does token-level accuracy differ between the two configurations, and where do the unidirectional errors concentrate (beginning, middle, or end of the sequence)? Could a 1D convolution solve this problem with fewer parameters? What kernel size would be needed?

Task 3: The Founding Titan's Attention Core — "Do Not Compress History. Read It."

Three walls breached. One fortress remained.

ZEKE-0's final defense was his masterpiece. He had compressed his entire historical database — every hidden state, every encoded signal — into a single fixed-length vector. A perfect information bottleneck. Every decoder query received the same collapsed summary regardless of what context it actually needed. It was elegant and tyrannical: all of history reduced to one thought, then forced through a single door.

"An encoder without attention," Abhinav said quietly. He recognized the architecture. He knew its weakness like he knew his own heartbeat. "The decoder is blind. One vector, one summary, used for every output position. The longer the sequence, the more information is lost. It cannot scale and it cannot focus."

"So we don't give it one vector," Prabanjan said from the floor. He was lying on his back, staring at the ceiling — his thinking posture. Suraj had long since stopped judging it. "We give it all of them. Every hidden state. And then we let the model learn, for each output position, which encoder states to actually look at."

"Dynamic context," Suraj said.

"Attention," Prabanjan confirmed. He did not get up. He was already working.

Problem Statement

The target function is the same centered window sum from Task 2b:

$$y_t = \sum_{j=\max(0,\, t-K)}^{\min(t+K,\, N-1)} x_j, \qquad K = 2, \quad N = 25$$

This time, the architecture must be an **RNN Encoder** with a **Bahdanau Attention Decoder**. Instead of collapsing the encoder output to a single context vector, the decoder queries all encoder hidden states at every step and computes a weighted combination.

The Encoder

Implement `Encoder(input_vocab, d_model, num_layers=1)` with:

- `nn.Embedding(input_vocab, d_model)`
- `nn.RNN(d_model, d_model, num_layers=num_layers, batch_first=True)`

The encoder forward pass takes x of shape (B, T) and returns the full sequence of hidden states H of shape (B, T, d_model). **All** intermediate states must be returned — do not discard them.

The Attention Mechanism

Implement `Attention(d_model)` with learnable projections `Wq`, `Wk` (`nn.Linear(d_model, d_model, bias=False)`) and a scoring vector `va` (`nn.Linear(d_model, 1, bias=False)`).

At each decoding step t, given query vector $h_t \in \mathbb{R}^d$ and all encoder states $H \in \mathbb{R}^{B \times T \times d}$:

$$e_{t,i} = \mathbf{v}_a^\top \tanh\big(W_q\, h_t \;+\; W_k\, h_i\big) \qquad \text{(energy score for encoder state } i\text{)} \tag{4}$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})} \qquad \text{(attention weight, softmax-normalized)} \tag{5}$$

$$c_t = \sum_{i=1}^{T} \alpha_{t,i}\; h_i \qquad \text{(dynamic context vector)} \tag{6}$$

The forward method receives `query_ht` of shape (B, d) and H of shape (B, T, d), and returns both $c_t \in \mathbb{R}^{B \times d}$ and $\alpha_t \in \mathbb{R}^{B \times T}$.

The Decoder

Implement `Decoder(output_vocab, d_model)` containing the `Attention` module and learned output projections `V` (`nn.Linear(d_model, output_vocab, bias=False)`) and `M_c` (`nn.Linear(d_model, output_vocab, bias=True)`).

The decoder loops over $t = 0, \ldots, T-1$. At each step:

1. Use $H[:, t, :]$ as the attention query
2. Compute context c_t and weights α_t via `Attention`
3. Compute output logits:
$$\ell_t \;=\; V \cdot H[:, t, :] \;+\; M_c \cdot c_t$$
4. Collect all ℓ_t into logits of shape $(B, T, \texttt{output\_vocab})$
5. If `return_attention=True`, also collect all α_t into shape (B, T, T)

Do **not** apply softmax to the final logits.

The Full Model

Implement `RNNAttentionModel(K, d_model=128)` that:

1. Sets `output_vocab` $= 9 \times (2K + 1) + 1$
2. Instantiates `Encoder(10, d_model)` and `Decoder(output_vocab, d_model)`
3. In the forward pass: encodes x to get H, then decodes H to get logits
4. Passes `return_attention` through to the decoder when specified

System Constraints

- Default $K = 2$; dataset: 10 000 sequences, 80/20 split
- Training: 10 epochs, batch size 256, Adam, `lr = 1e-2`, gradient clipping `max_norm = 1.0`
- Loss and evaluation: same as Task 1

Written Questions:

Q1. Visualize the attention weight matrix for several test samples as a heatmap (decoder query position t on the y-axis, encoder key position i on the x-axis). Does the learned attention pattern reveal that the model focuses on the correct K-neighborhood around each query position?

Q2. Compare this model's accuracy and convergence speed against Task 2b's bidirectional RNN on the same window-sum problem. Which reaches higher accuracy faster, and what does that tell you about the relative strengths of structural inductive bias (bidirectionality) versus learned dynamic focus (attention)?

Task 4: The LSTM — Abhinav's Final Weapon

ZEKE-0 was silent. His three walls had fallen. But deep inside his core fortress, the team found one last locked vault: a character-level sequence archive, centuries of encoded history compressed into addition strings, each one a test of whether a machine could truly understand sequential structure or was merely memorizing patterns.

"He built his whole architecture on the idea that sequences are hard," Abhinav said, reading the vault's header. "That compressing them loses information. That the bottleneck is unavoidable." He paused. "We already proved him wrong on attention. Now we go back to the beginning and prove it on everything else."

Prabanjan, who had been awake for four hours straight — a personal record — looked up from his keyboard. "The LSTM Vault. You kept it."

"It's still the right tool for this vault," Abhinav said. "Keras. TensorFlow. Encoder-decoder. Character-level addition. We crack every layer of it."

Suraj cracked his knuckles. "Let's finish this."

Part A: Addition as a Mapping Problem

The first layer treats addition as a plain regression: two integers in, their sum out. The goal is to understand what vanilla LSTM regression can and cannot do, before confronting the full sequence problem.

Task A1: Data Generation

Implement `random_sum_pairs(n_examples, n_numbers, largest)`:

- Generate `n_examples` samples, each with `n_numbers` random integers in $[1, \texttt{largest}]$
- Compute the sum of each sample
- Return normalized arrays X and y, dividing by `largest` $\times$ `n_numbers`

Also implement `invert_sum(value, n_numbers, largest)` to reverse the normalization.

Task A2: Build the LSTM Regression Model

Build a Keras `Sequential` model:

- `LSTM(10, input_shape=(n_numbers, 1))`
- `Dense(1)`
- Compile with `mean_squared_error` loss and `adam` optimizer

Input must be reshaped to `(n_examples, n_numbers, 1)`.

Task A3: Train the Model

Train for `n_epoch` epochs. Each epoch: generate a fresh batch with `random_sum_pairs`, reshape X, and call `model.fit(..., epochs=1, batch_size=n_batch, verbose=0)`. Store per-epoch loss in `history_a`.

Task A4: Evaluate

Evaluate on 100 new samples. Compute RMSE using `sklearn.metrics.mean_squared_error`. Print the first 20 predictions alongside ground-truth values.

Written Questions:

Q1. What RMSE did you achieve? Is the model learning the task correctly?

Q2. Could a simple MLP solve this same problem? Why or why not?

Part B: Addition as a True Seq2Seq Problem

"The real battle," Abhinav said, tapping the board, "is character-level. ZEKE-0 doesn't think in integers. He thinks in sequences. We have to match him."

The input is now a padded character string like `" 3+10"` and the output is the padded result string `"13"`. Every character is one-hot encoded. This is a full encoder-decoder pipeline.

Task B1: Convert Numbers to Padded Strings

Implement `to_string(X, y, n_numbers, largest)`:

- Join each input list (e.g. `[3, 10]`) into the string `"3+10"`, left-padded with spaces to fixed length $\texttt{n\_numbers} \times \lceil \log_{10}(\texttt{largest} + 1) \rceil + \texttt{n\_numbers} - 1$
- Convert each sum to a right-padded string of length $\lceil \log_{10}(\texttt{n\_numbers} \times (\texttt{largest} + 1)) \rceil$

Task B2: Integer-Encode the Characters

Implement `integer_encode(X, y, alphabet)` mapping each character to its index in:

```
alphabet = ['0','1','2','3','4','5','6','7','8','9','+',' ']
```

Task B3: One-Hot Encode

Implement `one_hot_encode(X, y, n_classes)` converting each integer index to a binary vector of length `n_classes` with `1` at the index position.

Task B4: Full Data Pipeline

Implement `generate_data(n_samples, n_numbers, largest, alphabet)` chaining Tasks B1–B3.

Task B5: Build the Encoder-Decoder LSTM

Build the seq2seq model:

```
Input: (n_in_len, n_chars)
  LSTM(100)                                    -> context vector (100,)
  RepeatVector(n_out_len)                      -> (n_out_len, 100)
  LSTM(50, return_sequences=True)
  TimeDistributed(Dense(n_chars, activation='softmax'))
Output: (n_out_len, n_chars)
```

Compile with `categorical_crossentropy` loss and `adam` optimizer.

Task B6: Train and Evaluate

Train for `n_epoch` epochs, storing accuracy per epoch in `history_b_acc` and loss in `history_b_loss`. Evaluate on 100 held-out samples using exact character-sequence match accuracy.

Written Questions:

Q3. Why does the seq2seq model need one-hot encoding while Part A used normalized floats?

Q4. What does `RepeatVector` do architecturally? Why is it needed?

Q5. What accuracy did you achieve? Were there specific digit patterns the model failed on?

Part C: Extensions and Analysis

Task C1: MLP Baseline

Build a Keras MLP that takes the same normalized input as Part A and predicts the sum. Compare RMSE with the Part A LSTM.

Q6. Does the MLP match or beat the LSTM? What does this reveal about whether the mapping formulation is truly a sequence problem?

Task C2: Scale Up

Retrain the seq2seq model with `n_numbers = 3` and `largest = 99`. Report accuracy.

Q7. How did accuracy change? What architectural or training changes helped?

Task C3: Loss Curve Analysis

Plot three subplots side by side: Part A loss, Part B loss, Part B accuracy. Label all axes.

Q8. Describe the shape of the loss curves. Did either model show signs of overfitting? Which converged faster?

Task C4: Error Analysis

Generate 500 test samples. Identify all incorrect predictions and analyze patterns. Print 10 representative failure cases.

Q9. What patterns did you observe in the failure cases? Is the model making systematic errors (e.g. consistently wrong in a specific digit position)?

Part D (Bonus): Subtraction Seq2Seq

Extend the seq2seq model to learn subtraction ($a - b$ where $a \geq b$). Update the alphabet to include `'-'` if needed. For extra challenge, support mixed operations ($+$ and $-$ randomly assigned per sample).

Q10 (Bonus). Did the model successfully learn subtraction? What was the hardest part of extending the approach?

Task	Model	Metric	Your Result
Part A	Stacked LSTM	RMSE	
Part C1	MLP Baseline	RMSE	
Part B	Seq2Seq LSTM	Exact-match Accuracy (%)	
Part C2	Seq2Seq (scaled)	Exact-match Accuracy (%)	
Part D	Subtraction Seq2Seq	Exact-match Accuracy (%)	

Epilogue: After the Rumbling

At 06:12 AM, the terminal printed something new:

```
ZEKE-0:  All walls breached.  Sequence pipeline:  RESTORED. Status:  NOMINAL.
```

The three of them sat in silence for a moment.

Suraj exhaled slowly and closed his laptop. That was the most he ever said when something went right.

Abhinav leaned back, arms behind his head, watching the attention heatmap on the screen — clean diagonal bands, each decoder step precisely focused on its correct K-neighborhood. The model had not been told where to look. It had learned. By itself. From data.

He thought about that for a while.

"Not bad," he said.

From the beanbag, a quiet sound. Prabanjan had fallen asleep, a Frooti still balanced in his hand at a physically improbable angle, a faint smile on his face.

He had figured all of it out. He just never looked like he was trying.

Suraj put a blanket over him without being asked.

Outside, the sun rose over Wall Maria for the first time in years without a shadow behind it.

■